



Aprendizado de Máquinas Profundo — Redes Neurais

Projeto 2 — Redes Neurais Recorrentes

Tradução Automática de Datas com Mecanismo de Atenção

Alunos:

Amanda Rehder Cedro — RA: 253507

Elisa Amorim Da Costa — RA: 277145

Iuri Santos Oliveira — RA: 194610

Pedro Galhardo Barbosa — RA: 277218

Professor: João Florindo

Universidade Estadual de Campinas

Instituto de Matemática, Estatística e Computação Científica

2026

Sumário

1 Introdução **3**

2 Fundamentação Teórica **4**

2.1 Redes Neurais Recorrentes (RNNs) 4

2.2 Long Short-Term Memory (LSTM) 4

2.3 Mecanismo de Atenção 4

2.3.1 Encoder Bidirecional 5

2.3.2 Cálculo dos Pesos de Atenção 5

3 Metodologia **6**

3.1 Dataset e Pré-processamento 6

3.2 Arquitetura do Modelo 6

3.3 Implementação Computacional 8

3.3.1 Camadas compartilhadas e o papel de cada uma 8

3.3.2 O loop de T_y saídas 9

3.4 Treinamento 10

4 Resultados **11**

4.1 Comparação entre os Cenários de Treinamento 11

4.2 Influência da pré-ativação e pós-ativação 11

4.3 Mapas de Atenção 13

5 Discussão **16**

5.1 Impacto do Número de Épocas 16

5.2 Papel do Número de Unidades de Pré e Pós-Ativação 16

5.3 Mapas de Atenção 17

5.4 Possíveis Melhorias 17

6 Conclusão **18**

1 Introdução

A tradução automática neural (*Neural Machine Translation* — NMT) é uma das áreas em que algoritmos de aprendizado profundo mais evoluíram nos últimos anos. O sucesso de ferramentas como o Google Tradutor, sustentado por arquiteturas cada vez mais sofisticadas, ilustra o impacto prático dessas técnicas no processamento de linguagem natural (PLN).

As primeiras abordagens neurais para tradução utilizavam a arquitetura *encoder-decoder* baseada em Redes Neurais Recorrentes (RNNs) e, especificamente, em *Long Short-Term Memory* (LSTM) [4]. Nesse paradigma, o encoder comprime toda a sequência de entrada em um único vetor de contexto fixo, que é então utilizado pelo decoder para gerar a sequência de saída. Embora funcional para sequências curtas, essa abordagem apresenta um gargalo crítico: toda a informação precisa estar contida em um vetor de dimensão fixa, o que se torna especialmente problemático para sequências longas.

O mecanismo de atenção, proposto por Bahdanau et al. [2], revolucionou esse cenário ao permitir que o decoder acesse dinamicamente todos os estados ocultos do encoder a cada passo de decodificação, ponderando a relevância de cada posição da entrada. Isso eliminou o gargalo do vetor fixo e melhorou substancialmente o desempenho em tarefas de tradução. Posteriormente, a arquitetura Transformer [5], baseada exclusivamente em mecanismos de atenção (*multi-head self-attention*), substituiu as conexões recorrentes e passou a dominar o estado da arte em NMT e em modelos de linguagem de larga escala como BERT e GPT.

Este projeto propõe uma tarefa de tradução simplificada, porém pedagogicamente rica: a conversão de datas escritas em formato humano (em inglês), como ‘‘`17th of October, 2023`’’ ou ‘‘`tuesday october 17 2023`’’, para o formato padronizado `YYYY-MM-DD`. Apesar de aparentemente simples, a tarefa exige que o modelo aprenda a reconhecer diferentes padrões linguísticos e a alinhar corretamente os elementos da entrada (dia, mês, ano) com as posições correspondentes na saída. Trata-se, portanto, de um caso de uso concreto e interpretável do mecanismo de atenção.

2 Fundamentação Teórica

2.1 Redes Neurais Recorrentes (RNNs)

Uma Rede Neural Recorrente processa sequências de entrada $x = (x^{\langle 1 \rangle}, x^{\langle 2 \rangle}, \ldots, x^{\langle T_x \rangle})$ de forma iterativa, mantendo um estado oculto $a^{\langle t \rangle}$ que acumula informações dos passos de tempo anteriores. A cada passo t, o estado oculto é atualizado por:

$$a^{\langle t \rangle} = \tanh\left(W_{aa}\, a^{\langle t-1 \rangle} + W_{ax}\, x^{\langle t \rangle} + b_a\right) \tag{1}$$

e a predição é dada por:

$$\hat{y}^{\langle t \rangle} = \text{softmax}\left(W_{ya}\, a^{\langle t \rangle} + b_y\right) \tag{2}$$

Apesar da elegância conceitual, as RNNs básicas sofrem do problema de *vanishing gradient*: durante o treinamento via *backpropagation through time* (BPTT), os gradientes tendem a desaparecer exponencialmente ao se propagar por muitos passos de tempo, dificultando o aprendizado de dependências temporais de longo alcance [3].

2.2 Long Short-Term Memory (LSTM)

A LSTM [1] resolve o problema do *vanishing gradient* introduzindo um estado de célula $c^{\langle t \rangle}$ e três mecanismos de portas que controlam o fluxo de informação:

$$\Gamma_f^{\langle t \rangle} = \sigma\left(W_f \left[a^{\langle t-1 \rangle}, x^{\langle t \rangle}\right] + b_f\right) \quad \text{(porta de esquecimento)} \tag{3}$$

$$\Gamma_u^{\langle t \rangle} = \sigma\left(W_u \left[a^{\langle t-1 \rangle}, x^{\langle t \rangle}\right] + b_u\right) \quad \text{(porta de atualização)} \tag{4}$$

$$\tilde{c}^{\langle t \rangle} = \tanh\left(W_c \left[a^{\langle t-1 \rangle}, x^{\langle t \rangle}\right] + b_c\right) \quad \text{(candidato)} \tag{5}$$

$$c^{\langle t \rangle} = \Gamma_f^{\langle t \rangle} * c^{\langle t-1 \rangle} + \Gamma_u^{\langle t \rangle} * \tilde{c}^{\langle t \rangle} \quad \text{(célula)} \tag{6}$$

$$\Gamma_o^{\langle t \rangle} = \sigma\left(W_o \left[a^{\langle t-1 \rangle}, x^{\langle t \rangle}\right] + b_o\right) \quad \text{(porta de saída)} \tag{7}$$

$$a^{\langle t \rangle} = \Gamma_o^{\langle t \rangle} * \tanh\left(c^{\langle t \rangle}\right) \tag{8}$$

onde $*$ denota o produto de Hadamard e $[a^{\langle t-1 \rangle}, x^{\langle t \rangle}]$ a concatenação dos vetores. As portas criam um "caminho de gradiente" direto através do estado de célula (Equação 6), que é atualizado por operações aditivas, permitindo que gradientes fluam eficientemente por muitos passos de tempo.

2.3 Mecanismo de Atenção

O mecanismo de atenção [2] permite que o decoder acesse dinamicamente todos os estados ocultos do encoder. Em modelos *encoder-decoder* sem atenção, toda a informa-

ção da entrada é comprimida no último estado oculto do encoder — um gargalo que o mecanismo de atenção elimina.

2.3.1 Encoder Bidirecional

O encoder utiliza uma LSTM bidirecional (BiLSTM), que processa a sequência tanto da esquerda para a direita quanto da direita para a esquerda. Para cada posição t', o estado oculto do encoder é a concatenação dos estados das duas direções:

$$a^{\langle t' \rangle} = \left[\overrightarrow{a}^{\langle t' \rangle}; \overleftarrow{a}^{\langle t' \rangle} \right] \in \mathbb{R}^{2n_a} \tag{9}$$

O uso da BiLSTM é motivado pelo fato de que a interpretação de um caractere pode depender tanto do contexto à esquerda quanto à direita. Por exemplo, o caractere "0" em uma data pode fazer parte do ano "09" (2009) ou do dia "01", e o contexto bidirecional auxilia na correta identificação.

2.3.2 Cálculo dos Pesos de Atenção

A cada passo de decodificação t, os pesos de atenção $\alpha^{\langle t,t' \rangle}$ são calculados como:

1. Uma pequena rede *feedforward* computa as energias de alinhamento entre o estado anterior do decoder $s^{\langle t-1 \rangle}$ e cada estado do encoder $a^{\langle t' \rangle}$:

$$e^{\langle t,t' \rangle} = f_{\text{att}} \left(s^{\langle t-1 \rangle}, a^{\langle t' \rangle} \right) \tag{10}$$

2. Os pesos são obtidos normalizando as energias via softmax:

$$\alpha^{\langle t,t' \rangle} = \frac{\exp(e^{\langle t,t' \rangle})}{\sum_{k=1}^{T_x} \exp(e^{\langle t,k \rangle})} \tag{11}$$

3. O vetor de contexto é a soma ponderada dos estados do encoder:

$$\text{context}^{\langle t \rangle} = \sum_{t'=1}^{T_x} \alpha^{\langle t,t' \rangle} \, a^{\langle t' \rangle} \tag{12}$$

A intuição por trás desse mecanismo é direta: ao predizer cada caractere de saída, o modelo deve "focar" nos caracteres da entrada mais relevantes para aquela posição. Os pesos $\alpha^{\langle t,t' \rangle}$ quantificam exatamente essa relevância, e são visualizáveis através dos mapas de atenção.

3 Metodologia

3.1 Dataset e Pré-processamento

O dataset consiste em 10.000 pares de datas (formato humano, formato máquina). Cada caractere da entrada é mapeado para um índice inteiro via dicionário `human_vocab` (37 símbolos distintos) e representado como vetor *one-hot* de dimensão 37. A sequência de entrada é truncada ou preenchida (*padding*) para comprimento fixo $T_x = 30$. A saída utiliza `machine_vocab` (11 símbolos: dígitos 0–9 e o hífen), com comprimento fixo $T_y = 10$, correspondente ao formato `YYYY-MM-DD`.

As dimensões dos tensores resultantes são: $X_{oh} \in \mathbb{R}^{10000\times30\times37}$ e $Y_{oh} \in \mathbb{R}^{10000\times10\times11}$.

Utilizou-se a função `tf.keras.utils.set_random_seed` (combinada com a configuração de determinismo operacional do framework) para fixar as sementes globais, garantindo a reprodutibilidade dos experimentos e a padronização dos resultados de saída do código.

3.2 Arquitetura do Modelo

O modelo segue a arquitetura *encoder-decoder* com atenção. A Figura 1 apresenta o diagrama esquemático da arquitetura implementada, mostrando os três blocos principais: o encoder BiLSTM (pré-atenção), o mecanismo de atenção e o decoder LSTM (pós-atenção).

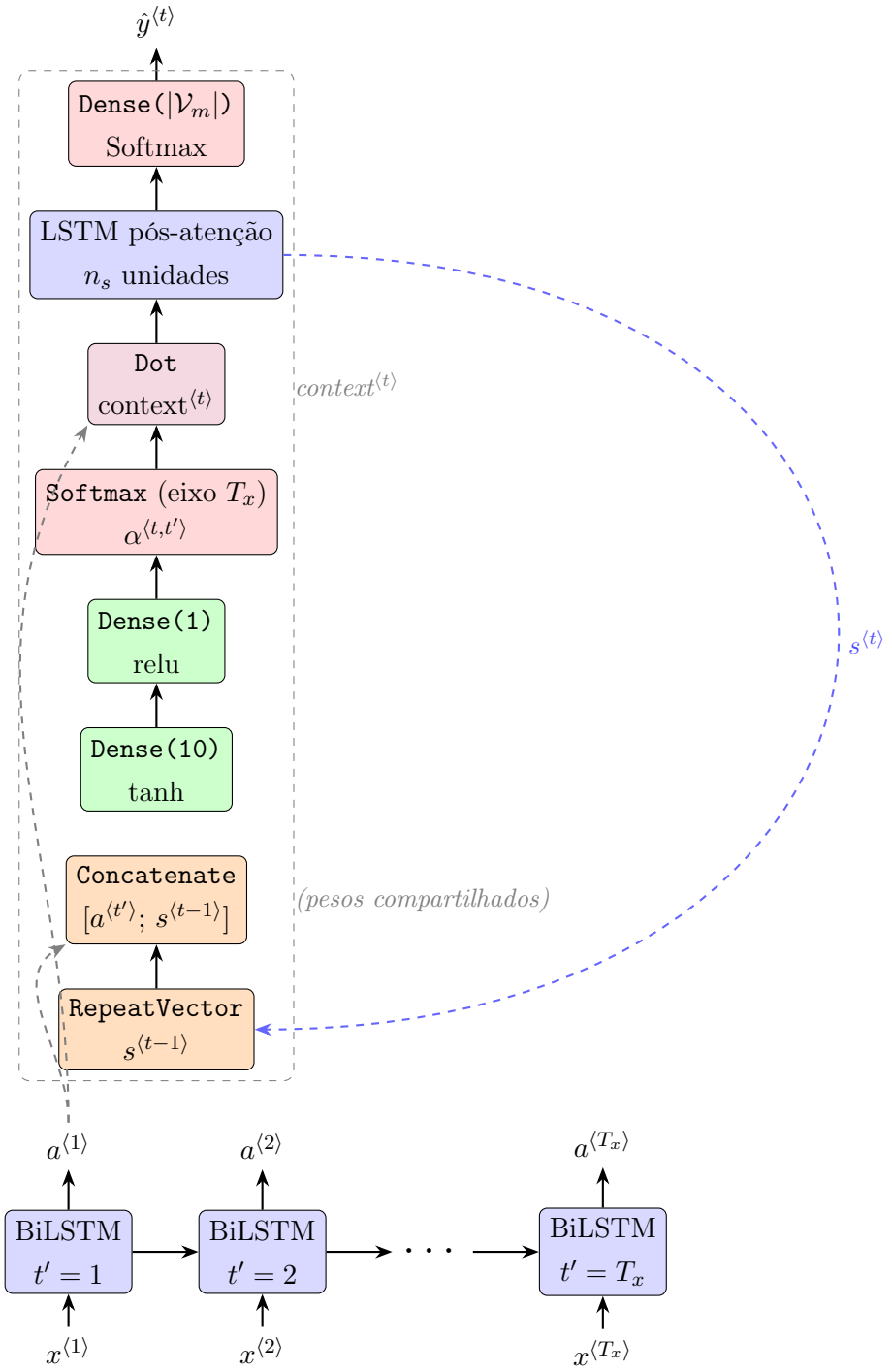


Figura 1: Diagrama esquemático da arquitetura implementada.

O encoder BiLSTM processa toda a sequência de entrada e produz os estados $a^{\langle t' \rangle}$. A cada passo t do decoder, o mecanismo de atenção (bloco tracejado) computa o vetor de contexto ponderado, que alimenta a LSTM pós-atenção para gerar o caractere $\hat{y}^{\langle t \rangle}$. O estado $s^{\langle t \rangle}$ é realimentado como $s^{\langle t-1 \rangle}$ no passo seguinte (seta azul tracejada).

Os três blocos principais são descritos a seguir:

1. **Encoder**: BiLSTM com $n_a = 32$ unidades por direção, produzindo estados ocultos $a^{\langle t' \rangle} \in \mathbb{R}^{64}$ para cada uma das $T_x = 30$ posições de entrada.

2. **Mecanismo de atenção**: A função `one_step_attention` implementa as Equações (11) e (12) por meio de uma rede com duas camadas *Dense* (10 unidades com ativação `tanh` e 1 unidade com ativação `relu`), seguidas de softmax ao longo do eixo temporal da entrada e produto interno (camada `Dot`) com os estados do encoder.

3. **Decoder**: LSTM com $n_s = 64$ unidades (configuração base), que recebe o vetor de contexto a cada passo e atualiza os estados $s^{\langle t \rangle}$ e $c^{\langle t \rangle}$. Uma camada *Dense* com softmax sobre 11 classes produz a distribuição de probabilidade para o caractere de saída em cada passo $t = 1, \ldots, T_y$.

O modelo possui ao todo 52.960 parâmetros treináveis na configuração base ($n_a = 32$, $n_s = 64$).

3.3 Implementação Computacional

Esta seção detalha os principais componentes de implementação do modelo em Keras, descrevendo *como* cada bloco da arquitetura foi construído em código.

3.3.1 Camadas compartilhadas e o papel de cada uma

Uma característica central da implementação é que as camadas do mecanismo de atenção são **definidas uma única vez fora do loop** e reutilizadas em todos os $T_y = 10$ passos do decoder. Isso garante o compartilhamento de pesos entre os passos de tempo — condição necessária para que o modelo generalize o alinhamento:

Listing 1: Definição das camadas compartilhadas do mecanismo de atenção.

```
repeator     = RepeatVector(Tx)        # replica s_{t-1} por Tx vezes
concatenator = Concatenate(axis=-1)    # concatena a e s_prev ao longo do ultimo eixo
densor1      = Dense(10, activation="tanh")  # camada de energia intermediaria
densor2      = Dense(1,  activation="relu")  # energia escalar por posicao
activator    = Activation(softmax_axis1,
                          name='attention_weights')  # normaliza em Tx
dotor        = Dot(axes=1)             # produto ponderado: alphas . a
```

`RepeatVector(Tx).` O estado anterior do decoder $s^{\langle t-1 \rangle} \in \mathbb{R}^{n_s}$ é um vetor com uma única dimensão temporal. Para que possa ser concatenado com os T_x estados do encoder $a \in \mathbb{R}^{T_x \times 2n_a}$, é necessário *replicá-lo* T_x vezes ao longo do eixo do tempo, produzindo um tensor de forma $(m,\, T_x,\, n_s)$. Isso é feito pelo `RepeatVector`.

`Concatenate(axis=-1).` Após a replicação, concatena-se $a \in \mathbb{R}^{m \times T_x \times 2n_a}$ com $s_{\text{prev}} \in \mathbb{R}^{m \times T_x \times n_s}$ ao longo do último eixo (features), resultando em um tensor de forma $(m,\, T_x,\, 2n_a + n_s)$. Esse tensor contém, para cada posição de entrada, a informação local do encoder combinada com o estado atual do decoder.

Camadas `Dense` (energias). O tensor concatenado é processado por duas camadas densas aplicadas *posição a posição* (pois o Keras aplica camadas densas no último eixo de tensores 3D):

1. `densor1`: projeta para 10 unidades com ativação `tanh`, capturando interações não-lineares entre encoder e decoder;
2. `densor2`: projeta para 1 unidade com ativação `relu`, produzindo um escalar de energia $e^{\langle t,t' \rangle}$ por posição de entrada — tensor resultante de forma $(m,\, T_x,\, 1)$.

`Activation(softmax_axis1).` A função `softmax` é aplicada ao longo do eixo T_x (eixo 1), normalizando as energias de todas as T_x posições de entrada para que somem 1. O resultado são os pesos de atenção $\alpha^{\langle t,t' \rangle}$, que formam uma distribuição de probabilidade sobre as posições de entrada para cada passo de decodificação t.

`Dot(axes=1).` Calcula o produto interno entre os pesos de atenção $\alpha \in \mathbb{R}^{m \times T_x \times 1}$ e os estados do encoder $a \in \mathbb{R}^{m \times T_x \times 2n_a}$ ao longo do eixo T_x (eixo 1). O resultado é o **vetor de contexto** $\text{context}^{\langle t \rangle} \in \mathbb{R}^{m \times 1 \times 2n_a}$ — uma combinação linear ponderada de todos os estados do encoder, em que as posições mais relevantes para o passo t recebem maior peso.

3.3.2 O loop de T_y saídas

O decoder opera em um loop explícito de $T_y = 10$ iterações. Em cada passo t, o estado oculto atual s é usado pelo mecanismo de atenção para calcular o vetor de contexto, que por sua vez alimenta a LSTM pós-atenção para produzir o próximo estado e o caractere de saída:

Listing 2: Loop do decoder com mecanismo de atenção.

```
for t in range(Ty):
    # 1. Mecanismo de atencao: calcula context a partir de s (
       estado atual)
    context = one_step_attention(a, s)
```

```
4
5       # 2. LSTM pos-atencao: recebe context e atualiza s e c
6       s, _, c = post_activation_LSTM_cell(context, initial_state=[
            s, c])
7
8       # 3. Camada densa: prediz o caractere de saida no passo t
9       out = output_layer(s)
10      outputs.append(out)
```

Note que a LSTM pós-atenção recebe **apenas o vetor de contexto** como entrada (não a saída anterior do decoder). Isso significa que o decoder não realiza *teacher forcing* implícito na arquitetura: a informação sobre o que foi gerado anteriormente chega indiretamente via os estados s e c, que carregam o histórico acumulado. Essa escolha simplifica o grafo computacional e elimina a necessidade de gerenciar a sequência de saída durante o treinamento.

Ao final do loop, o modelo é instanciado com as três entradas (`X`, `s0`, `c0`) e a lista de T_y tensores de saída:

Listing 3: Instanciação do modelo Keras com múltiplas saídas.

```
model = Model(inputs=[X_in, s0, c0], outputs=outputs)
```

O Keras compila esse grafo como um único modelo, permitindo que a *backpropagation* flua através de todos os T_y passos simultaneamente durante o treinamento. Cada posição de saída contribui com sua própria *cross-entropy*, e a perda total é a soma das T_y perdas individuais.

3.4 Treinamento

O modelo foi treinado com o otimizador Adam (lr = 0,005, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, *weight_ decay* = 0,01) e função de perda *categorical cross-entropy* aplicada a cada uma das T_y saídas. O tamanho do *batch* foi de 100 exemplos. Os estados iniciais do decoder s_0 e c_0 foram inicializados com zeros.

Foram avaliados três cenários:

- **Modelo pré-treinado**: pesos fornecidos pelo professor (`model.h5`);
- **1 época**: treinamento do zero por 1 época;
- **50 épocas**: treinamento do zero por 50 épocas.

Adicionalmente, foram testadas cinco configurações distintas de (n_a, n_s), todas treinadas por 20 épocas, para avaliar a influência do número de unidades no desempenho final.

4 Resultados

4.1 Comparação entre os Cenários de Treinamento

A Tabela 1 apresenta as predições dos três cenários para sete exemplos de datas com formatos variados.

Tabela 1: Predições dos três modelos para exemplos de datas com diferentes formatos de entrada.

Entrada	**Pré-treinado**	**1 época**	**50 épocas**	**Correto?**
3 May 1979	1979-05-33	1987-01-04	1979-05-03	Parcial / Não / Sim
5 April 09	2009-04-05	2013-04-04	2009-04-05	Sim / Não / Sim
21th of August 2016	2016-08-20	2000-02-22	2016-08-03	Parcial / Não / Parcial
Tue 10 Jul 2007	2007-07-10	2000-02-22	2007-07-10	Sim / Não / Sim
Saturday May 9 2018	2018-05-09	2013-02-04	2018-05-09	Sim / Não / Sim
March 3 2001	2001-03-03	2003-06-22	2001-03-03	Sim / Não / Sim
1 March 2001	2001-03-01	2000-02-22	2001-03-01	Sim / Não / Sim

Este padrão da Tabela 1 nos mostra que o modelo de 1 época não convergiu e suas predições ignoram a entrada, repetindo saídas quase fixas como "2000-02-22". Já o modelo de 50 épocas se aproxima bastante do pré-treinado, acertando a maioria dos formatos regulares, o que mostra que o mecanismo de atenção aprende sozinho o alinhamento entrada-saída dado tempo suficiente de treinamento.

Um adendo importante é que nem o modelo de 50 épocas nem o pré-treinado acertam o caso mais irregular "21th of August 2016", o que sugere que o erro não é falta de treinamento, mas uma limitação ligada à baixa frequência desses padrões no dataset, reforçando que aumentar a diversidade dos dados tende a ajudar mais do que aumentar o número de épocas. .

4.2 Influência da pré-ativação e pós-ativação

A Tabela 2 e a Figura 2 apresentam os resultados para cinco configurações distintas de pré-ativação e pós-ativação, todas treinadas por 20 épocas.

Tabela 2: Desempenho por configuração de n_a (pré-atenção) e n_s (pós-atenção) após 20 épocas.

n_a	n_s	**Loss final**	**Acurácia final**
16	32	0,0430	1,0000
32	32	0,1375	1,0000
32	64	0,0350	1,0000
64	64	0,0171	1,0000
64	128	0,0112	1,0000

Todas as cinco configurações vistas na Tabela 2 atingem acurácia média próxima de 1,0000 após 20 épocas, o que já era esperado dado o vocabulário pequeno e a regularidade do problema. A métrica que realmente diferencia as configurações é o loss final, sendo que, aumentar n_a e n_s tende a reduzir o erro residual, como no contraste entre $n_a = 16, n_s = 32$ (loss 0,0430) e $n_a = 64, n_s = 128$ (loss 0,0112) que é consistente com n_a controlando a riqueza da representação contextual e n_s a capacidade de memória do decodificador.

O caso com $n_a = 32$ e $n_s = 32$ chama atenção e oferece um importante *insight* teórico. Apesar de possuir um número bruto de parâmetros maior do que a configuração mínima ($n_a = 16, n_s = 32$), esse modelo apresentou uma convergência nitidamente pior, mantendo um valor de *loss* superior ao longo de quase todo o treinamento.

Esse fenômeno comprova que um decodificador com capacidade de memória reduzida ($n_s = 32$) é incapaz de processar com eficiência um vetor de contexto excessivamente rico ou complexo proveniente de um encoder de maior dimensão ($n_a = 32$), indicando que o balanço entre n_a e n_s importa mais do que seus valores absolutos. Cria-se, portanto, um gargalo informacional onde a representação oculta expandida se perde na etapa de decodificação por falta de capacidade do estado recorrente pós-atenção. Esse resultado evidencia empiricamente que o equilíbrio dimensional e a harmonia estrutural entre as componentes de pré e pós-ativação importam muito mais para a dinâmica de aprendizado do que o aumento puramente quantitativo de parâmetros da rede.

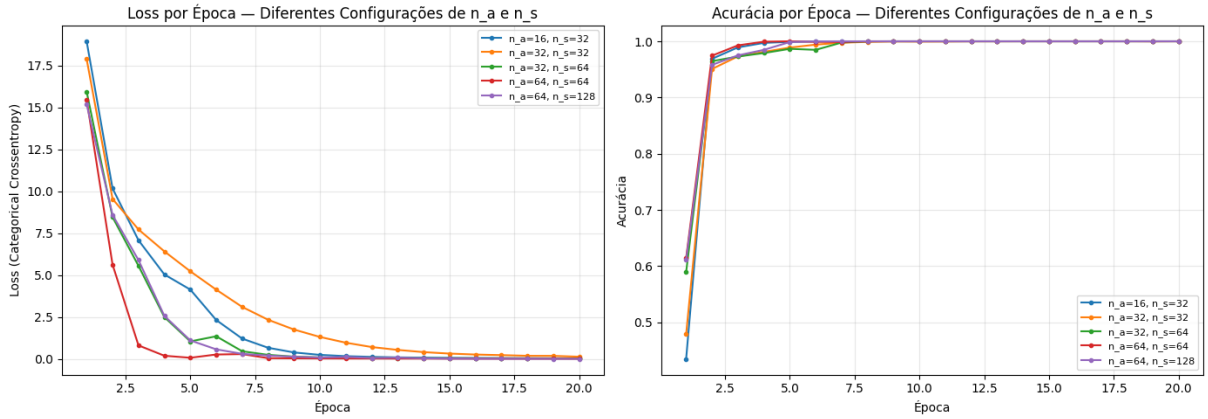


Figura 2: Curvas de perda (esquerda) e acurácia média (direita) por época para as cinco configurações de n_a (pré-atenção) e n_s (pós-atenção) após 20 épocas.

A Figura 2 mostra que a acurácia satura rapidamente e pouco diferencia as configurações (todas ultrapassando 0,95 já nas primeiras épocas). O loss, por outro lado, separa bem os modelos: a curva vermelha ($n_a = 64, n_s = 64$) converge mais rápido, enquanto a laranja ($n_a = 32, n_s = 32$) mantém o loss mais alto durante praticamente todo o treinamento.

As curvas com $n_a = 64$ apresentam os menores loss finais, mas o ganho de $n_s = 64$ para $n_s = 128$ é pequeno, sugerindo retornos decrescentes e que n_a pode ser o fator mais limitante. O desempenho pior de $n_a = 32, n_s = 32$ frente a $n_a = 16, n_s = 32$, apesar de mais parâmetros, indica que a combinação de hiperparâmetros importa mais que o tamanho total do modelo.

4.3 Mapas de Atenção

A Figura 3 apresenta os mapas de atenção gerados pelo modelo de 50 épocas para três exemplos. Cada célula da matriz representa o peso $\alpha^{\langle t,t' \rangle}$ entre o caractere de saída na posição t (eixo vertical) e o caractere de entrada na posição t' (eixo horizontal). Cores mais intensas indicam maior atenção.

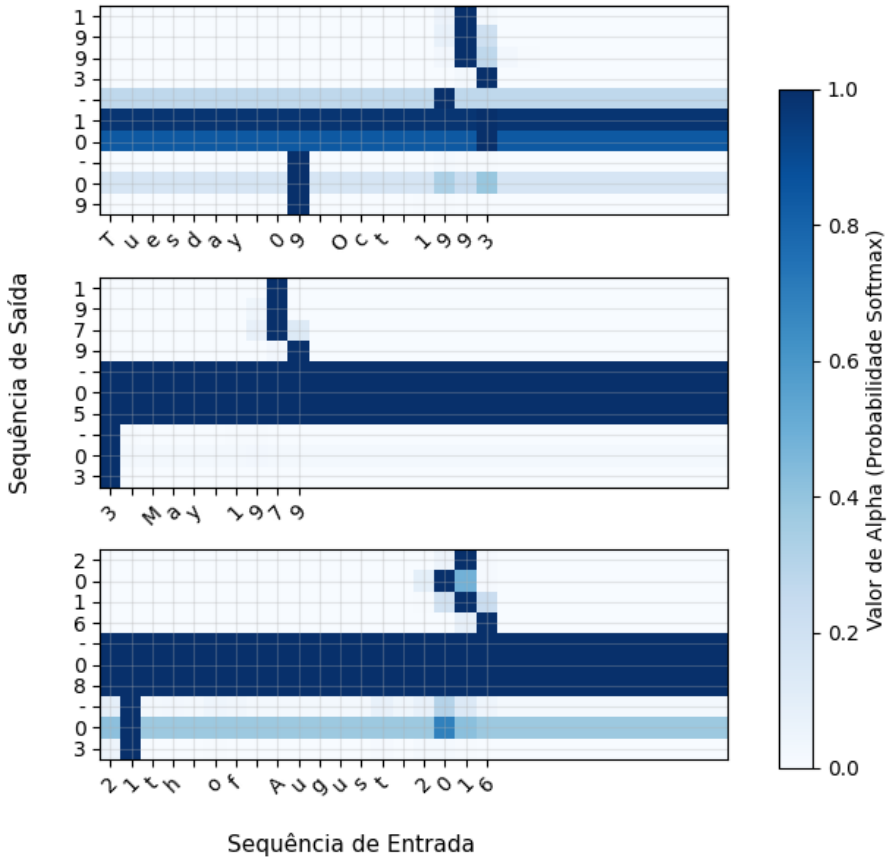


Figura 3: Mapas de atenção para as entradas, sendo o eixo vertical correspondente aos caracteres preditos na saída e o horizontal aos caracteres da entrada.

Esta Figura mostra que o modelo aprendeu a focar sua atenção nos trechos corretos da entrada para gerar cada caractere de saída, sem depender de palavras irrelevantes como o dia da semana ou o mês por extenso. No exemplo do meio ("3 May 1979"), por exemplo, os caracteres do ano na saída concentram quase toda a atenção em "1979", e o mês em "May", formando uma diagonal de blocos bem definidos, obtendo um comportamento esperado de um mecanismo de atenção bem treinado.

Ainda assim, o erro do dia (visíveis também na Tabela 1) aparece refletido no mapa "21th of August 2016", onde a atenção sobre o dia é mais difusa, distribuída entre vários caracteres em vez de concentrada, o que sugere que o modelo tem mais dificuldade em isolar a posição exata do dia quando ele aparece com sufixo ordinal ("21th"). Isso é coerente com a hipótese de que o erro é estrutural, ou seja, quando a atenção não converge para uma posição única, a predição do caractere correspondente também se torna menos confiável.

As limitações do modelo em cenários com alta variabilidade sintática tornam-se evidentes ao analisar o caso irregular `‘‘21th of August 2016’’`. Para essa entrada, o modelo de 50 épocas gerou a saída incorreta `2016-08-03`, falhando na decodificação do dia (predizendo `03` em vez de `21`).

Essa falha reflete-se perfeitamente na estrutura matemática do mapa de atenção cor-

respondente: ao contrário da diagonal nítida observada nos casos regulares, a distribuição dos pesos de alinhamento $\alpha^{\langle t,t' \rangle}$ para os dois últimos caracteres da saída mostra-se dispersa e difusa (*blurred*) ao longo da sequência de entrada. O mecanismo de atenção falhou em convergir para uma posição única, distribuindo a probabilidade Softmax de forma quase uniforme entre o número `21`, o sufixo ordinal `th` e o espaçamento adjacente. Do ponto de vista conceitual, essa entropia elevada na matriz de atenção indica que o decoder recebeu um vetor de contexto ruidoso e ambíguo, justificando mecanicamente a perda de confiabilidade e o erro na predição dos dígitos finais do dia.

5 Discussão

5.1 Impacto do Número de Épocas

Os resultados da Tabela 1 deixam evidente que uma única época de treinamento é completamente insuficiente para que o modelo aprenda os padrões da tarefa. Com loss de 15,42 após a primeira época, os pesos ainda se encontram longe de qualquer mínimo útil da função objetivo, e as predições são essencialmente aleatórias. O salto de desempenho ocorre a partir da segunda época: observando o log de treinamento das 50 épocas, a acurácia média já supera 90% na época 2 e atinge valores próximos a 100% entre as épocas 5 e 10. As épocas subsequentes servem principalmente para refinar os pesos e estabilizar a loss (que cai abaixo de 0,02 nas épocas finais), sem ganhos expressivos de acurácia.

A comparação entre o modelo pré-treinado e o de 50 épocas revela desempenhos similares. O único erro residual observado é o do pré-treinado para `3 May 1979` (`1979-05-33`), em que o dígito da unidade do dia é predito incorretamente. O modelo de 50 épocas acerta esse caso. Esse tipo de erro no dígito mais à direita do campo de dia sugere que essa posição específica é a mais difícil de generalizar, possivelmente por depender de combinações menos frequentes no dataset.

5.2 Papel do Número de Unidades de Pré e Pós-Ativação

Os experimentos com diferentes configurações de (n_a, n_s) revelam um resultado importante: para esta tarefa específica, o problema é suficientemente simples para ser resolvido por modelos bastante compactos. A configuração mínima testada ($n_a = 16$, $n_s = 32$, $\sim$22 k parâmetros) alcançou acurácia de 100% com loss de 0,0375, resultado competitivo com configurações substancialmente maiores.

A hipótese para o underperformance da configuração mais larga ($n_a = 64$, $n_s = 128$) é que, com aproximadamente 156 mil parâmetros para apenas 10.000 exemplos de treinamento, o modelo tende ao sobreajuste (*overfitting*), e o espaço de parâmetros maior torna a otimização mais difícil para a taxa de aprendizado utilizada. O pico de loss na época 13 (visível na Figura 2) é consistente com um fenômeno de "esquecimento catastrófico" local, possivelmente associado a uma atualização de gradiente que deslocou excessivamente alguns pesos. Esse tipo de instabilidade poderia ser mitigado com *gradient clipping* ou com redução da taxa de aprendizado.

Em geral, para tarefas de mapeamento de padrões fixos como a conversão de datas, arquiteturas moderadas ($n_a = 32$, $n_s = 64$) oferecem um bom equilíbrio entre capacidade representacional e facilidade de otimização.

5.3 Mapas de Atenção

Os mapas de atenção confirmam que o modelo aprendeu alinhamentos intuitivos e interpretáveis. Em todos os exemplos analisados, a atenção é aproximadamente local: ao predizer cada campo da saída (ano, separador, mês, separador, dia), o modelo foca nos caracteres de entrada que correspondem àquele campo. Para entradas mais curtas e de formato simples (e.g., `‘‘3 May 1979’’`), o padrão de atenção é quase monótono — a atenção avança da esquerda para a direita à medida que a saída também avança. Para entradas mais longas e complexas (e.g., `‘‘21th of August 2016’’`), a atenção é mais distribuída, indicando que o modelo precisa integrar informação de múltiplas posições para resolver ambiguidades introduzidas por termos funcionais como `‘‘of’’` e sufixos como `‘‘th’’`.

Vale notar que os pesos de atenção para as posições de *padding* (após o fim real da sequência de entrada) são sistematicamente baixos, indicando que o modelo aprendeu a ignorar os caracteres de preenchimento. Isso demonstra a capacidade do mecanismo de atenção de focar seletivamente na informação relevante.

5.4 Possíveis Melhorias

- **Dropout**: a adição de regularização entre as camadas LSTM reduziria o risco de sobreajuste, especialmente para configurações maiores;
- **Beam Search**: durante a inferência, a busca em feixe exploraria múltiplas hipóteses de decodificação, potencialmente eliminando os erros residuais observados em casos com sufixos ordinais;
- **Multi-head Attention**: a substituição do mecanismo de atenção simples pelo *multi-head attention* dos Transformers permitiria ao modelo capturar múltiplos padrões de alinhamento simultaneamente;
- **Early Stopping**: encerrar o treinamento quando a loss de validação para de diminuir evitaria épocas desnecessárias e ajudaria a identificar o ponto ótimo de treinamento;
- **Aumento de dados**: gerar variantes adicionais de formatos de data tornaria o modelo mais robusto a entradas não vistas.

6 Conclusão

Este projeto implementou e avaliou um modelo de tradução automática de datas textuais para o formato `YYYY-MM-DD` utilizando uma arquitetura *encoder-decoder* com BiLSTM e mecanismo de atenção. Os resultados demonstraram que o modelo converge rapidamente, atingindo acurácia superior a 99% em poucas épocas de treinamento.

A comparação entre os três cenários evidenciou que 1 época de treinamento é completamente insuficiente, enquanto 50 épocas produzem resultados competitivos com o modelo pré-treinado. Os experimentos com diferentes valores de n_a e n_s mostraram que modelos compactos são capazes de resolver esta tarefa com eficiência, e que o aumento excessivo de capacidade pode introduzir instabilidade e prejudicar a generalização.

Os mapas de atenção forneceram evidência qualitativa de que o modelo aprendeu alinhamentos interpretáveis entre posições de entrada e saída, confirmando que o mecanismo de atenção não apenas melhora o desempenho preditivo, mas também confere interpretabilidade ao processo de decisão do modelo.

Por fim, este trabalho reforça a compreensão prática das arquiteturas LSTM e do mecanismo de atenção, que constituem a base histórica sobre a qual foram construídos os Transformers — hoje dominantes em modelos de linguagem de larga escala como o GPT e o BERT.

Referências

[1] Hochreiter, S.; Schmidhuber, J. Long Short-Term Memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 1997.

[2] Bahdanau, D.; Cho, K.; Bengio, Y. Neural Machine Translation by Jointly Learning to Align and Translate. *arXiv preprint arXiv:1409.0473*, 2014.

[3] Goodfellow, I.; Bengio, Y.; Courville, A. *Deep Learning*. MIT Press, 2016. Disponível em: http://www.deeplearningbook.org.

[4] Cho, K. et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *arXiv preprint arXiv:1406.1078*, 2014.

[5] Vaswani, A. et al. Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.